IMPERIAL

pyop3: Extending the tensor abstraction to unstructured meshes

Connor J. Ward[1]
David A. Ham[1]
Jack D. Betteridge[2]

Mesh data ≠ ND arrays

N-dimensional (ND) arrays are the backbone of many areas of computational science. In particular they underpin machine learning frameworks such as PyTorch[3] and JAX[1].

Their popularity as an abstraction is due to two factors:

1. Hierarchically structured data is a natural programming model for many domains.
2. Tensor operations can be compiled to extremely high performance code.

However, ND arrays are not a suitable fit for calculations that involve unstructured meshes:

1. Mesh data can be associated with different topological entities (e.g. cells and/or vertices, fig. 1). This results in 'ragged' dimensions in the data structure that cannot be represented using typical ND arrays (fig. 2).
2. Mesh data are distributed in parallel and have complicated processes for ensuring global correctness.

pyop3 is this missing abstraction. It is a library that provides a numpy-like interface for interacting with unstructured mesh data. Just-in-time compilation is used to make the operations performant. Just like PyTorch and JAX for ML, and Firedrake for FEM[2], pyop3 provides a flexible, high-level, intuitive interface for building complex and performant programs, only this time for mesh-based simulation.

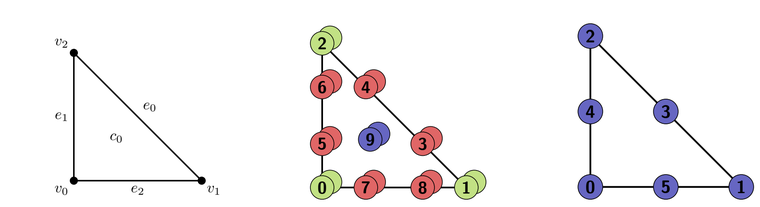

Figure 1: The Scott-Vogelius finite element pair $[P_3]^2 \oplus P_2^{\text{disc}}$ (middle and right) on a reference triangle (left). DoFs associated with vertices, edges and cells are shown in green, red and blue respectively. P_2^{disc} (right) is discontinuous, so all the DoFs are considered to belong to the cell.

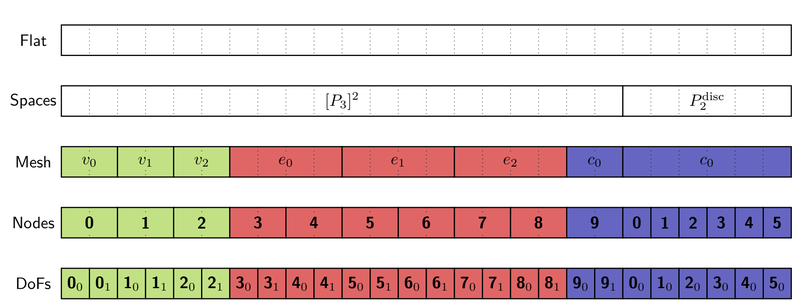

Figure 2: The data layout structure for a one-cell mesh of the finite element pair in fig. 1. It is not possible to capture all of this detail using an ND array.

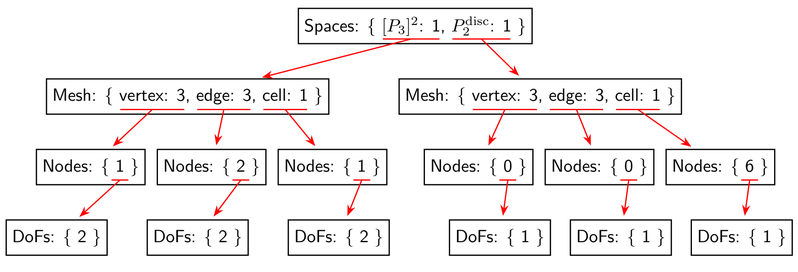

Figure 3: Graphical representation of an axis tree that describes the data layout shown in fig. 2. Unlike ND arrays, all information is captured by the abstraction.

The pyop3 language

pyop3 is a domain-specific language (DSL) that is embedded in Python. Users express operations using a numpy-like syntax (e.g. fig. 4) that are just-in-time compiled to high-performance code (fig. 5).

```
cell = mesh.cells.iter()
vec[closure(cell)]["v"][::2]
```

Figure 4: A non-trivial example of pyop3 numpy-like syntax in action: accessing the even numbered vertices around a cell. `closure(cell)` returns all entities that touch the cell, and `"v"` extracts only the vertices. Note both the similarity between this code and typical ND array usage with numpy, and also how indexing operations may be chained together.

PYTHON

```
loop(cell := mesh.cells.iter(),
     kernel(vec1[closure(cell)], vec2[cell]))
```

CODE GENERATION

C

```
void pyop3_kernel(array_0, array_1, array_2) {
  double t_0[3];
  double t_1[1];
  for (int32_t i_0 = 0; i_0 < ncells; ++i_0) {
    for (int32_t i_1 = 0; i_1 < 3; ++i_1)
      t_0[i_1] = array_0[array_1[3 * i_0 + i_1]];
    t_1[0] = 0.0;
    kernel(t_0, t_1);
    array_2[i_0] += t_1[0];
  }
}
```

Figure 5: Simplified example of the code generated from a pyop3 loop expression.

Operations in pyop3 are composable, and so it is possible to express non-trivial looping operations such as that shown in fig. 6.

```
loop(vertex := mesh.vertices.iter(),
     loop(cell := star(vertex, k=2).iter(),
          kernel(in_vec[closure(cell)], out_vec[vertex])))
```

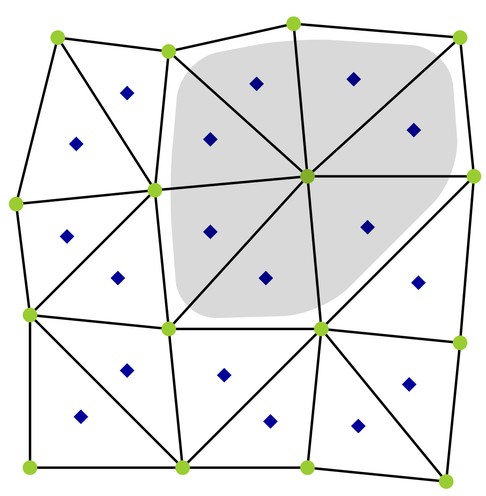

Figure 6: pyop3 loop expression describing an operation over a vertex patch. `star(vertex, k=2)` will return all cells that touch the current vertex. Note that the inner loop over cells is 'ragged' (has non-constant extent).

Representing mesh data

The core capability that ND arrays provide is the ability to address a block of memory using a set of indices (multi-index). They do this via an index expression of the form

$$(i_0, i_1, \ldots, i_n) \to \text{offset}$$

An ND array is composed of multiple axes and is addressed with an index per axis.

As an example, consider an array with shape `(3, 2)`. It has 2 axes with extents 3 and 2, and has offset expression $2i_0 + i_1$.

To represent mesh data without losing important topological information, pyop3 necessarily has a much richer abstraction for describing unstructured data layouts than simple ND arrays: the axis tree.

Like ND arrays, an axis tree is composed of multiple axes and has the ability to convert a multi-index into an offset. However, in pyop3, axes are composed of 1 or more components. This means that axes are able to branch.

By permitting branching, axis trees can represent the heterogeneous types of data needed for unstructured mesh calculations. For example, they can represent multi-field and multi-entity data layouts like that shown in fig. 2 (see fig. 3) where the offset expressions include

$$([P_3]^2, v_2, n_0, d_1) \to 5$$
$$(P_2^{\text{disc}}, c_0, n_3, d_0) \to 24$$

```
axes = AxisTree.from_nest({
  Axis("space", {"u": 1, "p": 1}): [
    {Axis("mesh", {"v": nverts, "e": nedges, "c": ncells): [
      Axis("dof", 1), Axis("dof", 2), Axis("dof", 1),
    ]},
    Axis("mesh", {"v": nverts, "e": nedges, "c": ncells): [...]
  ],
})
vec = op3.Dat.zeros(axes, dtype=float64)
```

Figure 7: Simplified pyop3 code constructing a vector (termed a 'dat') with a data layout (axis tree) similar to that shown in fig. 3.

Axis trees underpin pyop3's expressive syntax because complex indexing operations (e.g. figs. 4 to 6) are simply axis tree transformations.

An additional benefit of axis trees is that they decouple data semantics from implementation - one can permute axes of the axis tree to achieve a semantically equivalent data structure with a different data layout.

References

[1] James Bradbury, Roy Frostig, et al. JAX: composable transformations of Python+NumPy programs. Version 0.3.13. 2018.

[2] David A. Ham, Paul H. J. Kelly, et al. Firedrake User Manual. First edition. Imperial College London et al. 2023.

[3] Adam Paszke, Sam Gross, et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: Advances in Neural Information Processing Systems 32. 2019.

Affiliations

[1] Department of Mathematics, Imperial College London
[2] Devito Codes